

UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO
ARQUITETURA DE COMPUTADORES - 2026.2

PROFESSORA: Millena de Andrade Almeida Gomes Cavalcanti

ALUNOS: Túlio Lorca de Araujo Falcão, Henrique Barbosa Mendes, Lucas Thomaz de Santana.

ESPECIFICAÇÕES DA PRIMEIRA ENTREGA

Discorreremos sobre o projeto a partir do arquivo que executa o programa: [app.py](#).

A primeira parte do código trata da leitura e tratamento dos dados contidos no arquivo [entrada.py](#). A primeira chamada trata da função “decodificador()” do módulo [decodificador.py](#). Descreveremos a função, a seguir:

“Lê o arquivo JSON de entrada do simulador MIPS e extrai a lista de instruções codificadas em hexadecimal.”

Parâmetros: caminho_arquivo (str): caminho para o arquivo .json (padrão: 'entrada.json')

Retorna: list: Lista contendo as strings em Hexadecimal (ex.: ['0x02114020', ...])

A entrada de dados passa por uma etapa de leitura do arquivo json com garantia de suporte a caracteres UTF-8, retornando uma tupla com a chave ‘text’ e a lista com os dados extraídos, passando por uma validação que prevê caso o arquivo repassado não seja do tipo json. Esta tupla é carregada em uma variável no *main()* em *app*, chamada “instruções_hex”. É criada uma lista vazia chamada “instruções binárias” onde serão armazenadas as instruções hexadecimais convertidas em instruções binárias. Há uma filtragem no processo que converte o hexadecimal para inteiro binário, tirando o prefixo “0b” e depois garantindo que a string binária contenha 32 bits (preenchendo zeros à esquerda, se necessário), fazendo nova verificação se os arquivos de entrada são hexadecimais. Caso os dados passem no filtro, os binários são salvos na lista “instruções binárias”.

Essa lista entra como argumento da função “get_instrucao_mips()” que faz parte do módulo “[instrucoesMIPS.py](#)”. Este módulo traz um dicionário completo das instruções MIPS, contidas no documento que foi repassado na atividade. Cada instrução é inserida no dicionário “instrucoesMIPS”, contendo o nome da instrução, o tipo, o opcode e a função (funct). Esse dicionário é essencial para podermos comparar as instruções binárias coletadas e tratadas do arquivo de entrada para podermos saber qual a instrução a ser executada pelo *main()* do “[app.py](#)”. Dividimos o dicionário em instruções do tipo R, I, J e o caso especial que é a instrução “syscall”. A função “get_instrucao_mips()” carrega a lista “instrucoesBinarias” importada da *main()* e decodifica as palavras binárias de 32 bits para a sintaxe Assembly MIPS correspondente. É criada uma lista vazia chamada “desassembled” e depois o conteúdo da lista é “fatiado” para as seguintes variáveis: opcode, rs, rt, rd, shamt, funct, immediate_bin (caso não seja uma instrução do tipo ‘R’) e address_bin (mesma coisa). Cria-se uma variável vazia chamada ‘instrucao_encontrada’ que armazena o tipo da

instrução. É criado um 'for' para buscar o tipo da instrução baseado no opcode e funct fatiados anteriormente. Se o tipo da instrução não for encontrada, ele entra na lista 'desassembled' como uma instrução desconhecida. Como a instrução 'syscall' pode ser confundida com instruções do tipo R, por conta do mesmo opcode, é feita uma checagem, logo após. Daí, identificamos se a instrução é do tipo 'R' ou apenas é um 'syscall'. Depois, há checagens para instruções do tipo 'I' e 'J'. O resultado é colocado na lista desassembled, que é o retorno da função.

Este retorno é armazenado na variável 'código_desmontado', no módulo app, finalizando o segundo processo do executável:

“enviar a lista de strings binárias para a lógica central em [instrucoesMIPS.py](#).

O retorno é a lista de instruções formatadas em Assembly MIPS.”

A terceira parte no executável, trata da montagem da estrutura de saída exigida pela atividade. É exigido que a saída seja um json de objetos contendo hex, text, regs, mem e stdout.

É criada uma lista vazia chamada 'resultado_json'. É feito um 'for' com as variáveis 'instrucoes_hex' e 'codigo_desmontado', onde se alocam as informações 'hex_value' (instrução de entrada em hexadecimal) na variável 'hex' da saída do json, e o texto da instrução assembly MIPS, contida na variável 'asm_text' na variável 'text' do mesmo dicionário, deixando as outras variáveis vazias, conforme se pede nas especificações do projeto ('regs', 'mem' e 'stdout'). Esse dicionário originado pelo 'for' é salvo dentro da lista 'resultado_json'.

A quarta parte trata da geração do arquivo de saída json. Que gera uma mensagem de que tudo ocorreu corretamente no terminal e gera o arquivo 'saida.json' na pasta do projeto.